

RTL Design Sherpa

STREAM on the Genesys 2 The FPGA System: Harness, Builds, Flow -- Version 1.1

September 2026

Table of Contents

1 Document Information....9
1.1 Revision History....10
1.2 How to read this book....13
2 The System....14
2.1 What it is....14
2.1.1 Figure 1.1: The bench....14
2.2 The board....15
2.3 The two USB chips....16
2.4 The stack, top to bottom....17
2.5 What the host sees....18
3 The Harness....19
3.0.1 Figure 2.1: The harness block map....19
3.1 The host front-end and the bridge....20
3.2 The harness CSR....21
3.3 The run timer and the verdict....23
3.4 Stimulus and the memory model....24
3.5 The three instruments....25
4 The Address Map....26
4.0.1 Figure 2.2: Host address map....26
4.1 Bridge slave map....27
4.2 Harness CSR detail....29
4.3 By-name access....32

5 One RTL, Three Builds....33
5.1 The rule....33
5.1.1 Figure 3.1: One source tree, three bitstreams....34
5.2 Why three, and why not two....36
5.3 What the builds measured....37
5.4 The knobs behind the rows....41
6 Host Tools and Flow....43
6.1 The shared flow....43
6.2 The host layers....44
6.2.1 Figure 4.1: What each build measures, end to end....45
6.3 One program, two targets....46
6.4 Where the results go....47
7 STREAM against stream-g on This Board....49
7.1 The stream-g build....50
7.1.1 Figure 5.1: The stream-g DUT in the harness....51
7.2 The two bitstreams....53
7.3 The first stream-g bitstream....55
7.4 How the two designs were compared....56
7.4.1 Figure 5.2: The A/B, end to end....56
7.4.2 The simulation against the board....57
7.5 Single descriptor per channel (gcompare)....58
7.6 Descriptor chains (gcompare)....61
7.7 The size sweeps (gscale, gscale1)....62

7.8 The matrix at three descriptor sizes..... 64

7.9 Artifacts..... 65

List of Figures

Figure 1.1: The bench.....14
Figure 2.1: The harness block map.....19
Figure 2.2: Host address map.....26
Figure 3.1: One source tree, three bitstreams.....34
Figure 4.1: What each build measures, end to end.....44
Figure 5.1: The stream-g DUT in the harness.....49
Figure 5.2: The A/B, end to end.....54

List of Tables

Table 1: Table 0.1: Document information....9
Table 2: Table 0.2: Revision history....10
Table 3: Table 1.1: The part and what the builds use of it....15
Table 4: Table 1.2: The stack....17
Table 5: Table 2.1: Harness CSR, the registers that define a run....21
Table 6: Table 2.2: Stimulus and memory model....24
Table 7: Table 2.3: Bridge address map (from bridge_stream_mon_axil.toml)....27
Table 8: Table 2.4: harness_csr register map (base 0x0001_0000)....29
Table 9: Table 3.1: The builds....33
Table 10: Table 3.2: Sign-off builds....36
Table 11: Table 3.3: The knobs....40
Table 12: Table 4.1: Flow targets (from make/fpga_flow.mk)....42
Table 13: Table 4.2: Per-build host programs....43
Table 14: Table 4.3: Reports....46
Table 15: Table 5.1: The two Genesys 2 builds....51
Table 16: Table 5.2: The cores out of context on the xc7k325t-2 (128-bit data, 32-bit addresses, 8 outstanding each way, SRAM depth 256, shared engines, no AXI monitors, EXT addressing built; placed and routed at 100 MHz, 2026-10-24)....51
Table 17: Table 5.3: One descriptor per channel, 2026-10-25 (peak 1,600 MB/s)....56
Table 18: Table 5.4: Chains of descriptors per channel, 2026-10-25....59
Table 19: Table 5.5: STREAM / stream-g cycles over the size sweep (-- = not in the sweep)....60
Table 20: Table 5.6: Bytes per descriptor for stream-g to be within 10 %, 5 % and 1 % of STREAM....60
Table 21: Table 5.7: Cycles, STREAM / stream-g (stream-g relative to STREAM)....62

List of Waveforms

No waveforms in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

1 Document Information

Table 0.1: Document information

Field	Value
Title	STREAM on the Genesys 2 – The FPGA System
Version	1.4
Date	2026-10-24
Scope	The STREAM characterization and monitor-validation system: board, harness, the builds (mon, obs, perf and perf with stream-g), host flow, and the board-side STREAM / stream-g comparison
Not in scope	The STREAM core itself and the choice between STREAM and stream-g (HAS/MAS under `projects/components/dma-ip/stream/docs/`); the monitor and extended-addressing results (`reports/`, `stable/MANIFEST.md`)
Status	Current; written against the tree at the stream BUG-018 close (commit 022fa0e3e), run timer updated 2026-10-15

1.1 Revision History

Table 0.2: Revision history

Version	Date	Change
1.0	2026-09-29	First edition, after the component-level consolidation (one harness, one top, three parameter sets) and the mon and obs sign-off builds of September 2026.
1.1	2026-10-15	The run timer and CRC verdict are now `char_run_timer` from `utility-ip/misc`, shared with the RAPIDS harnesses: Figure 2.1 adds the block, Section 2 describes it, Figure 4.1's verdict names it, and the address map no longer calls `TIMER_CYCLES` free-running.
1.2	2026-10-20	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
1.3	2026-10-20	Section (h2) headings now open a new page, the pipeline default. No technical content

Version	Date	Change
		changed.
1.4	2026-10-24	Chapter 5, STREAM against stream-g on this board (stream TASK-020): the `STREAM_G` / `FW_WRITABLE` build, Figure 5.1 (the stream-g DUT's register path and sequencing in the harness), the two bitstreams' area and timing, the A/B method (Figure 5.2: one tree, one host program, the gcompare / gscale / gscale1 point profiles on the board and in simulation), every measured table, the first dead stream-g bitstream and the undriven-net scan it added to `make bitstream`, and the ILA build. Chapter 3 gains the stream-g build row and the `STREAM_G`, `FW_WRITABLE`, `STREAM_CHAR_ILA`, `STREAM_SHARED_ENGINES` and clock knobs; Figures 2.1 and 3.1 name the second DUT.
1.5	2026-10-25	STREAM's monitor egress is `monbus_axil4_group_1wr` (stream TASK-019): the

Version	Date	Change
		`stream_err` bridge slot is reserved (SLVERR), error records land in the top 4 KB of `comp_sram` (Section 2.2 address map, Section 2.1 capture), read back by `dump_monbus.py`.

1.2 How to read this book

Chapter 1 is the bench. Chapter 2 is the bitstream around the DUT, block by block, with the bridge address map every host tool depends on. Chapter 3 is why there are three bitstreams from one source and what each is for; it is the chapter to read before trusting a zero. Chapter 4 is the shared flow, the per-build host layers, and the one-program-two-targets rule. Chapter 5 is STREAM against stream-g on this board: the build, the method and the numbers. Every diagram is a Graphviz source under `assets/graphviz/`, rendered to PNG by `regenerate_all_graphviz.sh`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

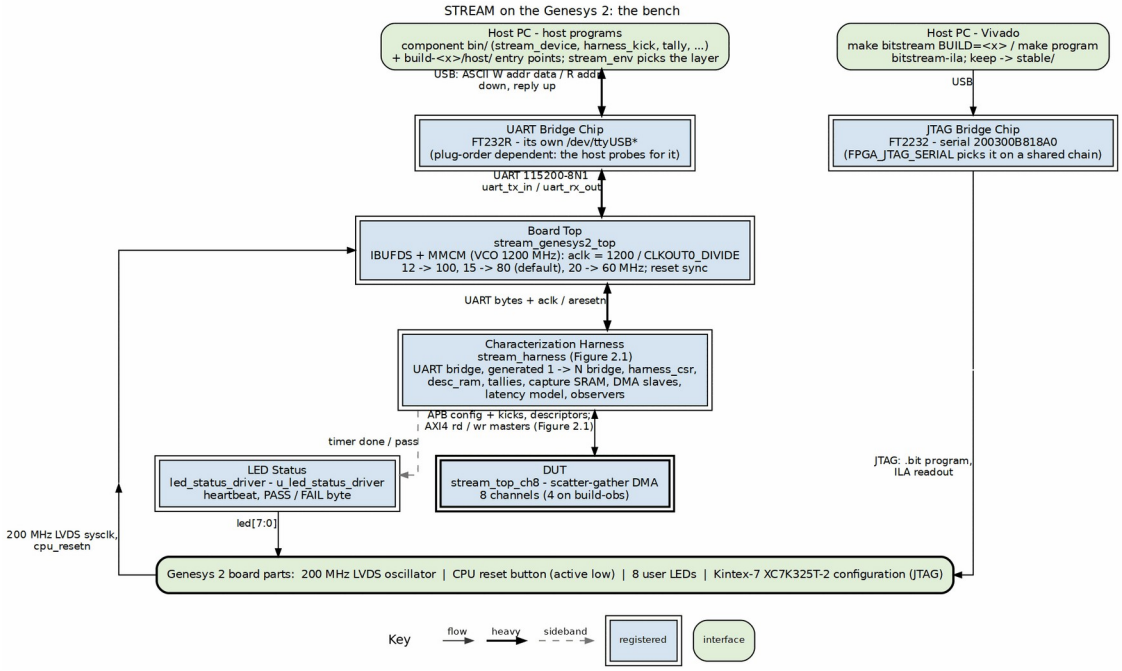
2 The System

2.1 What it is

The STREAM system on the Genesys 2 is two things sharing one bitstream source. It is a characterization harness: a synthesizable wrapper that feeds the STREAM scatter-gather DMA from an on-chip LFSR memory model, sinks its writes into a CRC checker, adds programmable memory latency, and counts every cycle of every interface. And it is a monitor-validation environment: the same wrapper carries the tally memories, capture SRAM and drains that let the host see every MonBus packet type from every agent in the DUT, whether the packets come from the in-core monitors or from interface observers standing outside the DUT.

Which of those it is on a given day is a question of which build is programmed. Chapter 3 is about that. The rest of this chapter is what does not change between them.

2.1.1 Figure 1.1: The bench



The bench: host, two USB chips, the FPGA

Source: 01_board_and_host.dot

2.2 The board

Digilent Genesys 2, Kintex-7 XC7K325T-2. The Nexys A7 version of this system (`stream_char_top`) was the original; the Genesys 2 top, `stream_genesys2_top`, adds an IBUFDS and an MMCM for the board's 200 MHz LVDS system clock and drops the seven-segment display for the eight user LEDs. Everything below the top is the same RTL on both boards.

The harness clock is a parameter, not a constant. The MMCM runs its VCO at 1200 MHz and the harness clock is 1200 divided by `CLKOUT0_DIVIDE`: 12 gives 100 MHz, 15 gives 80, 20 gives 60. `FPGA_CLK_HZ`, which sizes the UART divisor, the heartbeat and the timer, is derived from the same parameter, so the two can never disagree. The mon sign-off build ran at 60 MHz, which is where the full-cone monitor build closed with margin; the perf build is the one that cares about the clock, and it is small enough to run faster.

Table 1.1: The part and what the builds use of it

Resource	XC7K325T-2	The builds
Slice LUTs	203,800	mon sign-off 139,293 (68%); the two instruments together would need 217,761, which is why they are never built together (2026-10-03 re-measurement: 86,865 LUTs / 42.6%, WNS +1.914 ns; RTL has moved since the original sign-off)
Block RAM tiles	445	20 to 24 on the current reports
User LEDs	8	status bank; PASS and FAIL are one-byte patterns
Reset	red CPU reset button, active low	synchronized in the top

2.3 The two USB chips

The Genesys 2 carries JTAG on an FT2232 (serial `200300B818A0`, the one `FPGA_JTAG_SERIAL` names, because a Nexys A7 shares the chain in this lab) and the UART on a separate FT232R that enumerates as its own `/dev/ttyUSB*`. The host tools probe for the harness rather than pin a device node. The link is 115200-8N1 with the ASCII `W addr data` / `R addr` protocol, one 32-bit AXI4-Lite access per line.

2.4 The stack, top to bottom

Table 1.2: The stack

Layer	Module	Role
Board top	`stream_genesys2_top`	LVDS clock in, MMCM, reset sync, LEDs, the geometry generics
Harness	`stream_harness`	UART to AXI4-Lite bridge, the generated one-to-many bridge, harness CSRs, descriptor RAM, tallies, capture SRAM, DMA slaves, latency model, observers
DUT	`stream_top_ch8`	the STREAM DMA: APB configuration and kicks, eight channels (four on the obs build), one descriptor AXI4 master, read and write data masters, MonBus egress

The harness is component-level: one `stream_harness.sv`, one `stream_genesys2_top.sv`, one `harness_csr.sv`, one `axi_response_delay.sv`, one set of generated bridges, one Tcl set and one XDC, shared by every build. Every geometry default the harness uses comes from `stream_cfg_pkg`, which is the single source the board top and the simulation share. A literal in the harness is how the simulation and the board once drifted apart, with the perf characterization run at sixteen outstanding transactions against a board built at two.

2.5 What the host sees

One AXI4-Lite master, fanned out by a generated bridge to every slave in the harness: the STREAM APB window, the harness CSRs, the descriptor RAM, the MonBus error drains, the two tally memories and their configuration, the slave-monitor and observer register blocks, and the capture SRAM. Chapter 2 has the map. Every register the host touches is reached by name through a register map generated from `regs/harness_csr.rdl`; a hand-kept Python table used to exist, drifted, and declared ten registers the RTL did not decode, which three host tools then read as zeros off a running board.

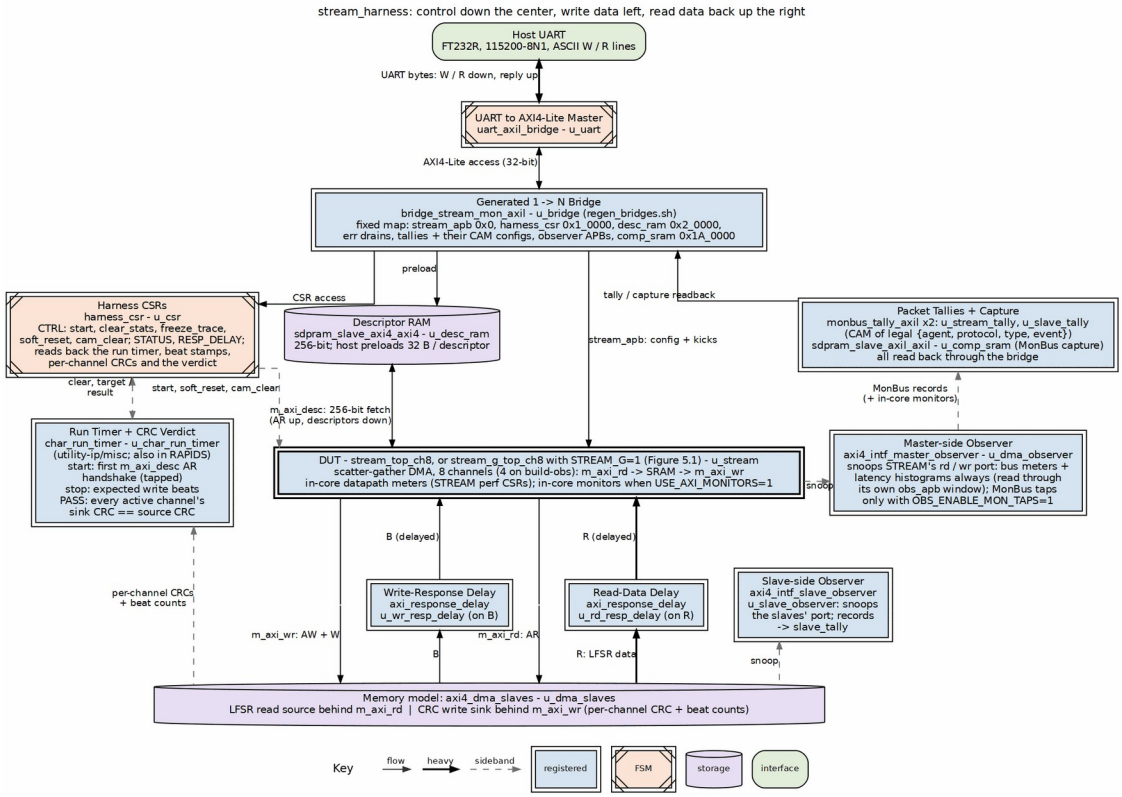
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 The Harness

`stream_harness` is the bitstream minus the pins. Unlike the RAPIDS harness, whose host front-end was moved inside it late, this one was built with the UART bridge and the CSRs inside from the start, so the cosim and the board have always run the same host programs over the same bytestream. Figure 2.1 is the block map.

3.0.1 Figure 2.1: The harness block map



The harness: front-end, stimulus, DUT, the three instruments

Source: 02_harness_blocks.dot

The DUT is `stream_top_ch8`, or `stream_g_top_ch8` when the perf build is made with `STREAM_G=1`: the same map, the same engines and masters, with the schedulers sequenced by firmware on a gatling core; its register path and sequencing structure are Figure 5.1, and the harness around it does not change.

3.1 The host front-end and the bridge

`uart_axil_bridge` turns the UART bytestream into one 32-bit AXI4-Lite master. A generated bridge (`bridge_stream_mon_axil`, from the bridge generator in `projects/components/fabric-gen-ip/bridge`) fans it out to every slave, converting protocol and width per port; it is regenerated by `bin/regen_bridges.sh` as the flow's PREBUILD step, so a slave added to the map is a generator input, not hand-written glue. The complete host address map – bridge slaves and the harness CSR register detail – is drawn in The Address Map.

Every MonBus-carrying AXI path is 64 bits wide. The descriptor read path is a direct 256-bit AXI4 from the DUT to `desc_ram`, bypassing the bridge, and the data masters are 128-bit into the pattern generator and CRC checker.

3.2 The harness CSR

`harness_csr` is the host's control and status surface, separate from the STREAM registers in the APB window. Its map is generated from `regs/harness_csr.rdl`; the RTL is hand-written against it and every external AXI4-Lite channel sits behind a skid buffer for timing.

Table 2.1: Harness CSR, the registers that define a run

Register	Role
`CTRL`	`start` (opens the measurement window), `clear_stats`, `freeze_trace` (stops capture writes), `soft_reset`, `cam_clear` (clears every CAM in the DUT's monitors and the tally)
`STATUS`	latched `stream_irq`, sticky `any_error`, sticky `trace_overflow`, `clear_busy` (poll to zero after `clear_stats` before the next capture)
`DBG_WR_PTR`, `DBG_OVERFLOW`	how much of the capture memory is written, and whether it overflowed
`CRC_RD_EXPECTED`, `CRC_WR_EXPECTED`, `CRC_WR_COMPUTED`	the source's pseudo-random CRC, the expected sink CRC, and what the sink actually wrote
(no meter registers)	`0x100`-`0x128`, where the retired harness bus meters were read back, is unallocated; the per-interface cycle classification is read from the observers' own register blocks (`OBS_STAT_SEL` /

Register	Role
	OBS_STAT_DATA) and from STREAM's in-core perf CSRs

`soft_reset` clears every instrument in the harness, which is what lets one programming run a long sequence of host programs without inheriting state; the runner hooks on the host issue it between programs.

3.3 The run timer and the verdict

`char_run_timer` (`u_char_run_timer`) times each run and judges it. It lives in `projects/components/utility-ip/misc/rtl/`. The two Genesys 2 RAPIDS harnesses use the same module and the fabric benchmark harness will, so every harness built on the LFSR source and CRC sink measures and judges a run the same way.

- **Start:** the first `m_axi_desc` AR handshake after a `TIMER_CTRL` clear, so the run begins with the first descriptor fetch.
- **Stop:** the sink's write-beat count reaching `TIMER_EXPECTED_BEATS`. `TIMER_CYCLES` freezes there, so the throughput is exact.
- **Pass:** sampled 5 cycles after the stop, once the last beat's CRC has settled. Every channel that ran must have a valid sink CRC equal to its source CRC; a channel that never ran neither passes nor fails. No channel is masked out of the verdict.
- **Stamps:** the cycle of the first and last read and write beat, so the host can separate steady-state engine throughput from descriptor fill and drain.

`TIMER_STATUS.pass` and the per-channel CRC registers in Table 2.1 are this block's outputs.

3.4 Stimulus and the memory model

Table 2.2: Stimulus and memory model

Block	Role
`axi4_dma_slaves`	an LFSR read-source pattern generator behind `m_axi_rd` and a write-sink CRC checker behind `m_axi_wr`; both accept the next transaction on the last beat of the current one (an earlier two-state version inserted a dead cycle per burst and cost the datapath a phantom 6 percent)
`axi_response_delay` x2	a programmable memory-latency model on the read and on the write path, the knob of the latency sweeps
`desc_ram`	the descriptor store the host preloads and the DUT fetches from directly

3.5 The three instruments

The harness carries three ways of measuring, and Chapter 3 is about which build has which.

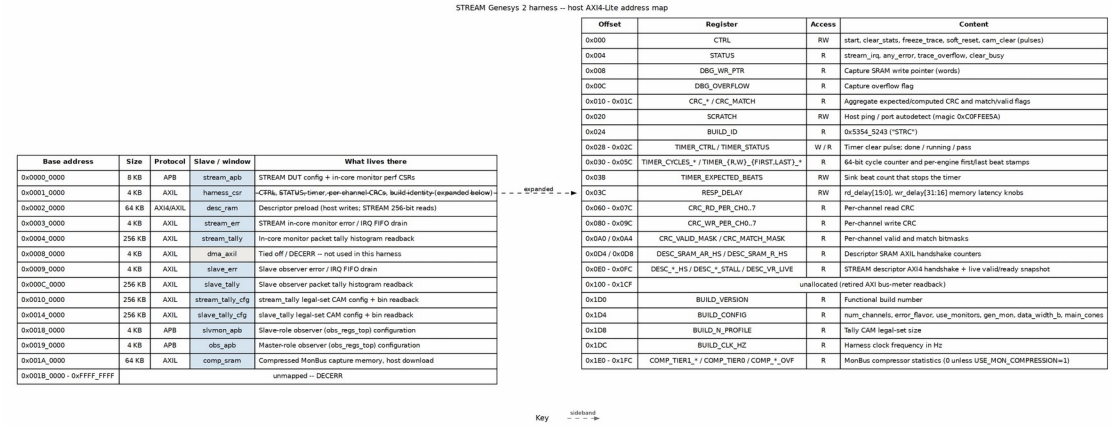
- **Bus meters**, always built, inside the interface observers and STREAM's in-core datapath (the harness's own `axi_bus_meter` blocks are retired). Per interface, every cycle of the window is productive, backpressure, starvation or idle. They sit outside the monitor gate on purpose: the perf build has no monitor and must still count. That separation regressed once, when the gate over-reached and the perf build reported zeros, and the monitor test now asserts the meters are non-zero in that flavour.
- **Tallies**, `monbus_tally_axil` twice. A direct count over every possible MonBus identity would be a 20,480-cell matrix of which about 245 tuples are legal; crossed with a 16-bit agent id it would be 99 percent empty. The tally instead loads the legal `{agent, protocol, packet type, event}` tuples into a CAM (`MON_N_PROFILE` slots, 32 on the sign-off builds) and counts densely, so the host can prove every agent emitted every packet type it may legally emit. `stream_tally` counts the in-core monitors, `slave_tally` the observers.
- **Capture**, `comp_sram`. The MonBus stream itself, compressed on the mon build, downloadable by the host and decoded against the cosim reference; the top 4 KB of `comp_sram` is the in-core group's ERROR window (STREAM's `monbus_axil4_group_1wr` writes `ERR_SELECT` records there and counts them in `MON_ERR_REC_COUNT`; `dump_monbus.py` reads them back); `slave_err` drains the observers' error / IRQ FIFO. The `stream_err` slot is reserved (SLVERR).

The two optional sources of packets are the DUT's in-core monitors (`USE_AXI_MONITORS`) and the interface observers standing on the DUT's masters and on the slaves (`OBS_ENABLE_MON_TAPS`). Both feed the same tally and capture machinery; they are never built together, and the reason is in Chapter 3.

4 The Address Map

The host sees one 32-bit AXI4-Lite master. `uart_axil_bridge` turns the ASCII `W addr data` / `R addr` lines into that master, and the generated `bridge_stream_mon_axil` fans it out to every slave in the harness. The bridge is generated from `rtl/bridges/configs/bridge_stream_mon_axil.toml`; the table below is the as-built map from that file. Anything not listed decodes to a tied-off `dma_axil` port or returns DECERR.

4.0.1 Figure 2.2: Host address map



Base address	Size	Protocol	Slave / window	What lives there
0x0000_0000	8 KB	APB	stream_apb	STREAM DUT config + in-core monitor perf CSRs
0x0001_0000	4 KB	AXIL	harness_csr	~~CTRL, STATUS, timer, per-channel CRCs, build-identity~~ (expanded below)
0x0002_0000	64 KB	AXI4/AXIL	desc_ram	Descriptor preload (host writes; STREAM 256-bit reads)
0x0003_0000	4 KB	AXIL	stream_err	STREAM in-core monitor error / IRQ FIFO drain
0x0004_0000	256 KB	AXIL	stream_tally	In-core monitor packet tally histogram readback
0x0008_0000	4 KB	AXIL	dma_axil	Tied off / DECERR -- not used in this harness
0x0009_0000	4 KB	AXIL	slave_err	Slave observer error / IRQ FIFO drain
0x000C_0000	256 KB	AXIL	slave_tally	Slave observer packet tally histogram readback
0x0010_0000	256 KB	AXIL	stream_tally_cfg	stream_tally legal-set CAM config + bin readback
0x0014_0000	256 KB	AXIL	slave_tally_cfg	slave_tally legal-set CAM config + bin readback
0x0018_0000	4 KB	APB	slvmon_apb	Slave-role observer (obs_regs_top) configuration
0x0019_0000	4 KB	APB	obs_apb	Master-role observer (obs_regs_top) configuration
0x001A_0000	64 KB	AXIL	comp_sram	Compressed MonBus capture memory, host download
0x001B_0000 - 0xFFFF_FFFF	unmapped -- DECERR			

Offset	Register	Access	Content
0x000	CTRL	RW	start, clear_stats, freeze_trace, soft_reset, cam_clear (pulses)
0x004	STATUS	R	stream_irq, any_error, trace_overflow, clear_busy
0x008	DBG_WR_PTR	R	Capture SRAM write pointer (words)
0x00C	DBG_OVERFLOW	R	Capture overflow flag
0x010 - 0x01C	CRC_* / CRC_MATCH	R	Aggregate expected/computed CRC and match/valid flags
0x020	SCRATCH	RW	Host ping / port autodetect (magic 0xC0FFEE5A)
0x024	BUILD_ID	R	0x5354_5243 ("STRC")
0x028 - 0x02C	TIMER_CTRL / TIMER_STATUS	W / R	Timer clear pulse; done / running / pass
0x030 - 0x05C	TIMER_CYCLES_* / TIMER_{R,W}_{FIRST,LAST}_*	R	64-bit cycle counter and per-engine first/last beat stamps
0x038	TIMER_EXPECTED_BEATS	RW	Sink beat count that stops the timer
0x03C	RESP_DELAY	RW	rd_delay[15:0], wr_delay[31:16] memory latency knobs
0x060 - 0x07C	CRC_RD_PER_CH0..7	R	Per-channel read CRC
0x080 - 0x09C	CRC_WR_PER_CH0..7	R	Per-channel write CRC
0x0A0 / 0x0A4	CRC_VALID_MASK / CRC_MATCH_MASK	R	Per-channel valid and match bitmasks
0x0D4 / 0x0D8	DESC_SRAM_AR_HS / DESC_SRAM_R_HS	R	Descriptor SRAM AXIL handshake counters
0x0E0 - 0x0FC	DESC_*_HS / DESC_*_STALL / DESC_VR_LIVE	R	STREAM descriptor AXI4 handshake + live valid/ready snapshot
0x100 - 0x1CF	unallocated (retired AXI bus-meter readback.)		
0x1D0	BUILD_VERSION	R	Functional build number
0x1D4	BUILD_CONFIG	R	num_channels, error_flavor, use_monitors, gen_mon, data_width_b, main_cores
0x1D8	BUILD_N_PROFILE	R	Tally CAM legal-set size
0x1DC	BUILD_CLK_HZ	R	Harness clock frequency in Hz
0x1E0 - 0x1FC	COMP_TIER1_* / COMP_TIER0 / COMP_*_OVF	R	MonBus compressor statistics (0 unless USE_MON_COMPRESSION=1)

Host address map

Source: 05_address_map.dot

4.1 Bridge slave map

Table 2.3: Bridge address map (from `bridge_stream_mon_axil.toml`)

Base	Size	Protocol	Slave	Purpose
0x0000_0000	8 KB	APB	stream_apb	STREAM DUT config and in-core monitor perf CSRs
0x0001_0000	4 KB	AXIL	harness_csr	Harness control/status/timer/CRC/build identity
0x0002_0000	64 KB	AXI4/AXIL	desc_ram	Descriptor preload: host writes 32-bit, STREAM reads 256-bit
0x0003_0000	4 KB	AXIL	stream_err	Reserved, SLVERR (was the in-core error FIFO drain; error records now land in comp_sram's top 4 KB, 0x001A_F000)
0x0004_0000	256 KB	AXIL	stream_tally	In-core monitor packet tally histogram readback
0x0008_0000	4 KB	AXIL	dma_axil	Tied off / DECERR – not used in this harness
0x0009_0000	4 KB	AXIL	slave_err	Slave observer error / IRQ FIFO drain

Base	Size	Protocol	Slave	Purpose
0x000C_0000	256 KB	AXIL	slave_tally	Slave observer packet tally histogram readback
0x0010_0000	256 KB	AXIL	stream_tally_cfg	stream_tally legal-set CAM config + bin readback
0x0014_0000	256 KB	AXIL	slave_tally_cfg	slave_tally legal-set CAM config + bin readback
0x0018_0000	4 KB	APB	slvmon_apb	Slave-role observer (obs_regs_top) configuration
0x0019_0000	4 KB	APB	obs_apb	Master-role observer (obs_regs_top) configuration
0x001A_0000	64 KB	AXIL	comp_sram	Compressed MonBus capture memory, host download

The two tally windows (stream_tally / slave_tally) are read-only for the host in normal use: records arrive directly from the observers or in-core monitors, and the host reads the resulting histogram. Their companion _cfg windows are where the host loads the legal {agent, protocol, type, event} CAM profile and where direct bin readback also lives.

4.2 Harness CSR detail

`harness_csr` occupies `0x0001_0000` and is the source of truth for run control, build identity, and the memory-latency knob. The map is generated from `regs/harness_csr_regs.rdl` and decoded by hand in `rtl/harness_csr.sv`; a test (`bin/tests/test_harness_regmap.py`) asserts every RDL register has a matching hardware decode.

Table 2.4: `harness_csr` register map (base `0x0001_0000`)

Offset	Register	Access	Content
0x000	CTRL	RW	`start`, `clear_stats`, `freeze_trace`, `soft_reset`, `cam_clear` pulses
0x004	STATUS	R	`stream_irq`, `any_error`, `trace_overflow`, `clear_busy`
0x008	DBG_WR_PTR	R	Capture SRAM write pointer (words)
0x00C	DBG_OVERFLOW	R	Capture overflow flag
0x010	CRC_RD_EXPECTED	R	Aggregate expected read-source CRC
0x014	CRC_WR_EXPECTED	R	Aggregate expected write-sink CRC
0x018	CRC_WR_COMPUTED	R	Actual CRC computed by write sink
0x01C	CRC_MATCH	R	`match[0]`, `valid[1]`
0x020	SCRATCH	RW	Host ping / port autodetect (magic

Offset	Register	Access	Content
			0xC0FFEE5A)
0x024	BUILD_ID	R	0x5354_5243 (“STRC”) – harness family
0x028	TIMER_CTRL	W	Timer clear pulse
0x02C	TIMER_STATUS	R	done, running, pass
0x030 / 0x034	TIMER_CYCLES_LO / HI	R	Run length in cycles, first descriptor AR to the expected write beat (char_run_timer; frozen at the end of the run, not free-running)
0x038	TIMER_EXPECTED_BEATS	RW	Sink beat count that stops the timer
0x03C	RESP_DELAY	RW	rd_delay[15:0], wr_delay[31:16] latency knobs
0x040 - 0x05C	TIMER_R_FIRST_* / TIMER_R_LAST_* / TIMER_W_FIRST_* / TIMER_W_LAST_*	R	Per-engine first/last beat cycle stamps
0x060 - 0x07C	CRC_RD_PER_CH0..7	R	Per-channel read CRC
0x080 - 0x09C	CRC_WR_PER_CH0..7	R	Per-channel write CRC
0x0A0 / 0x0A4	CRC_VALID_MASK / CRC_MATCH_MASK	R	Per-channel valid/match bitmasks
0x0D4 / 0x0D8	DESC_SRAM_AR_HS /	R	Descriptor SRAM

Offset	Register	Access	Content
	DESC_SRAM_R_HS		AXIL handshake counters
0x0E0 - 0x0FC	DESC_AR_HS, DESC_AR_STALL, DESC_R_HS, DESC_R_STALL, DESC_AW_HS, DESC_W_HS, DESC_B_HS, DESC_VR_LIVE	R	STREAM descriptor AXI4 handshake + live valid/ready snapshot
0x100 - 0x1CF	–	–	Unallocated (retired AXI bus-meter readback)
0x1D0	BUILD_VERSION	R	Functional build number
0x1D4	BUILD_CONFIG	R	num_channels, error_flavor, use_monitors, gen_mon, data_width_b, main_cones
0x1D8	BUILD_N_PROFILE	R	Tally CAM legal-set size
0x1DC	BUILD_CLK_HZ	R	Harness clock frequency in Hz
0x1E0 - 0x1FC	COMP_TIER1_A/B/C, COMP_TIER0, COMP_CAM_MISS, COMP_DELTA_TS_OVF, COMP_EVENT_DATA_OVF, COMP_ED_DELTA_OVF	R	MonBus compressor statistics (0 unless USE_MON_COMPRESSION=1)

4.3 By-name access

Host tools must resolve addresses by name, not by hardcoded offset:

- `bin/harness_addrs.py` `H("CTRL")` resolves harness CSR registers against `rtl/harness_csr_regmap.py` (generated from the RDL).
- `bin/stream_addrs.py` `A("SCHED_CONFIG")` resolves STREAM APB registers against the component-level `stream_regmap.py`.
- `bin/obs_addrs.py` `O("MON_CTRL")` resolves observer APB registers against `obs_regs_top_regmap.py`; use `base=SLAVE_OBS_APB_BASE` (`0x0018_0000`) for the slave-role observer.

This indirection is what keeps host scripts valid when a register moves inside its window.

5 One RTL, Three Builds

5.1 The rule

There is one harness, one board top, one CSR block, one latency model, one set of generated bridges, one Tcl set and one XDC, all at component level. `build-mon/`, `build-obs/` and `build-perf/` each hold a Makefile of variables and nothing else; the flow logic is the shared `make/fpga_flow.mk`. Which build you want is a question about what is under test, and the answer is a row in this table:

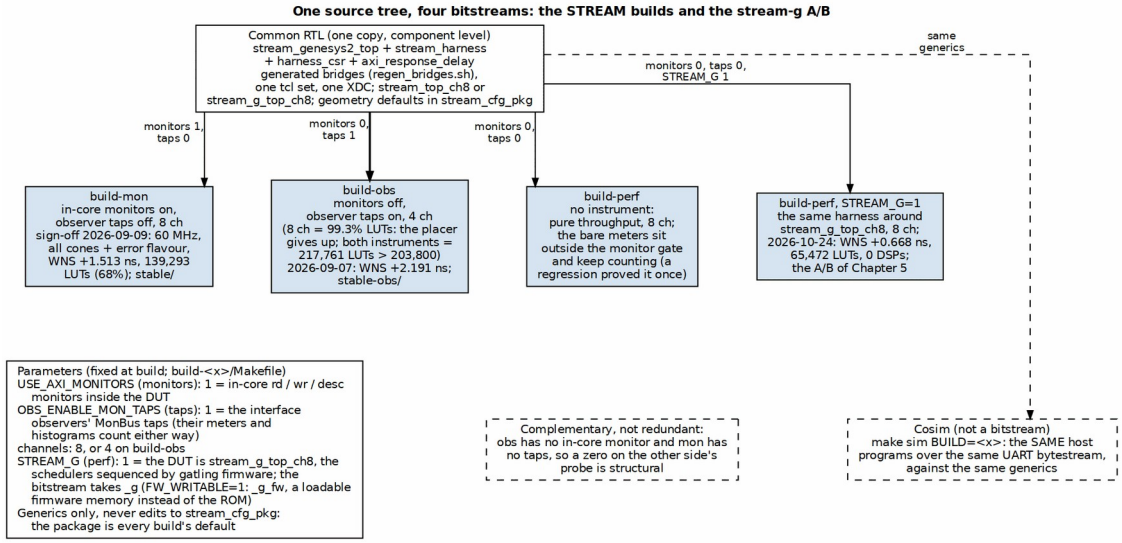
Table 3.1: The builds

Build	USE_AXI_MONITORS	OBS_ENABLE_MON_TAPS	Channels	Measures
`mon`	1	0	8	the in-core monitors: coverage, compression, error and fault probes
`obs`	0	1	4	the interface observers, watching the same DUT from outside it
`perf`	0	0	8	the datapath itself, with no instrument in the way
`perf`, `STREAM_G=1`	0	0	8	the same, around

Build	USE_AXI_MONITORS	OBS_ENABLE_MON_TAPS	Channels	Measures
				stream_g_top_ch8: the STREAM / stream-g A/B of Chapter 5 (FW_WRITABLE=1 for a host-loadable firmware memory)

Two near-identical harnesses, one per flavour, used to exist. They were retired because a variant that is a copy drifts: the board and the simulation once disagreed on the outstanding-transaction limit for exactly that reason. Now a variant is a parameter, the parameter is a generic handed to Vivado, and `stream_cfg_pkg` stays every build's default; editing the package to retarget one build silently retargets the other two.

5.1.1 Figure 3.1: One source tree, three bitstreams



One RTL, the knobs, the three builds

Source: 03_build_variants.dot

5.2 Why three, and why not two

The mon and obs builds are complementary, not redundant. They exercise the monitor code from two directions: the in-core monitors see transactions from inside the DUT, the observers see them on the interfaces outside it, and both push their packets through the same tally and capture machinery. So build-obs has no in-core STREAM monitor at all, and a zero on any `mon_*` probe there is structural; build-mon has its observer taps off, and a zero on an observer probe there is equally structural. Reading either zero as a defect has cost an afternoon before, which is why the Makefile says so in capitals.

They cannot be one build. With every monitor cone and both observers at eight channels the design needs 217,761 LUTs against the 203,800 on the part, and the placer never runs. Even the obs build alone does not fit at eight channels: it reached 99.33 percent of the LUTs and “could not place all instances”, so obs is pinned at four (six does not route either). The eight-channel design point is perf’s requirement; the obs channel count is incidental, not a traded-away requirement.

The perf build exists so that a number about the datapath is a number about the datapath. Its bus meters are instantiated outside the monitor gate and keep counting with every instrument removed; that arrangement is what lets the design fit a small part, and it is guarded by a test after it regressed once.

5.3 What the builds measured

Table 3.2: Sign-off builds

Build	Sign-off	Clock	Result
`mon`	2026-09-09, `stable/MANIFEST.md`	60 MHz	all cones plus the error flavour, compression built, 8 channels, `MON_N_PROFILE=32` : WNS +1.513 ns, 139,293 LUTs (68.35%), 0 failed nets; 512 packets in 516 slots on the board, 66.4% smaller than the raw encoding, CRC match, cosim reference within 4 points (2026-10-03 re-measurement: 66.0% / 1.02 slots/packet, within data-dependent noise; bit sha256 `aceec688043ad630879779ddae211814fcc4dc457365d7c06ec95b8a4f348e6b` , WNS +1.914 ns, 86,865 LUTs)
`obs`	2026-09-07, `stable-obs/MANIFEST.md`	as built	4 channels, 32-entry tally CAM, WNS +2.191 ns, archived the day a `make clean-all` destroyed the

Build	Sign-off	Clock	Result
			previous obs bitstream and it turned out obs had been synthesizing at 100.01 percent of the LUTs and placing by luck (2026-10-03 re-measurement: `build-obs/results/obs_board_2026-10-03.txt`, bit sha256 `60696ace63fed2c77ad4d3f1f511d06dd85801c9b5a9dac1d4078454192bfa5f`, WNS +3.925 ns, 73,809 LUTs; the observer instrument changed 2026-09-27, commit 78cddb5e2 — lite taps, no perf/debug cone, caps0 bit4/bit5 read 0)
`perf`	current reports on disk	as built	68,139 LUTs (33.4%), 24 BRAM tiles in `build-perf/fpga/reports`; the sweeps behind `reports/perf` and `reports/ext_addr`

Build	Sign-off	Clock	Result
			essing (2026-10-03 re-measurement: build-perf/results/perf_sweep_2026-10-03.{csv,json}, bit sha256 d26f9b7b3ac534a1507ab5a519e728d541b1045cf67000329fbedf77e3b9512c, WNS +1.175 ns, 64,305 LUTs)
perf	2026-10-24, build-perf/fpga/bitstream/stream_perf_genesys2_8ch.bit	100 MHz	the STREAM side of the Chapter 5 A/B: 66,221 LUTs (32.5 %), 26 BRAM, 64 DSP, WNS +0.790 ns, hold +0.032 ns; bit sha256 ff9a6b95ff7d58f091e77acd23c5ee8c175d4451e72b75bd060664dc8889452a; its 1 MB matrix cycle-identical to 2026-10-03
perf, STREAM_G=1	2026-10-24, build-perf/fpga/bitstream/stream_perf_genesys2_8ch_g.bit	100 MHz	stream-g, ROM firmware: 65,472 LUTs (32.1 %), 29 BRAM, 0 DSP, WNS +0.668 ns, hold +0.050 ns; bit sha256 b403115324623aa04d272c1710c5d0d35eec2bf157060d26

Build	Sign-off	Clock	Result
			`8300d0d9981687d1` ; 192 of 192 profile points and 120 of 120 matrix runs passing

The `stable/` directories hold exactly one build each, with the bitstream, the Vivado reports for that bitstream, the board results measured on it and a manifest naming the tree SHA it was built from. They exist because `make clean-all` in a build directory deletes tracked bitstreams and reports, and that has destroyed a verified bitstream more than once; `stable/` is a sibling of the build directories, outside that blast radius.

5.4 The knobs behind the rows

Table 3.3: The knobs

Knob	Where	What it selects
`STREAM_NUM_CHANNELS`	build Makefile	channel count: 8 for mon and perf, 4 for obs
`USE_AXI_MONITORS`	build Makefile, DUT generic	the in-core rd/wr/descriptor monitors, their CAMs, and the compression pipeline (gated on this flavour only)
`OBS_ENABLE_MON_TAPS`	build Makefile, harness generic	arm the observers' MonBus event taps; the observers' meters and histograms count without them
`MON_ERROR_FLAVOR`, `MON_CONES`	build Makefile	which monitor cones are built; the sign-off is the union including error
`MON_N_PROFILE`	build Makefile	tally CAM depth
`STREAM_SHARED_ENGINES`	build Makefile, DUT generic	1 (default): one descriptor engine fetches for every channel; 0: one per channel behind an arbiter (bitstream `_pce`)
`STREAM_G`, `FW_WRITABLE`	build-perf Makefile, harness define	`STREAM_G=1`: the DUT is `stream_g_top_ch8` (bitstream `_g`, `BUILD_CONFIG.STREAM_G=1`); `FW_WRITABLE=1`: its firmware memory is a block RAM the host loads through `FW_*` (bitstream `_g_fw`) instead of a ROM
`STREAM_CHAR_ILA`	`make bitstream-ila` (fpga/tcl/build_ila.tcl)	marks the host AXI4-Lite, STREAM APB, DUT-master and monitor-writer handshakes for an ILA; the ordinary builds carry no trace of it
`BOARD`, `A7_CLK_MHZ`	build Makefile	the target (genesys2, 8 channels at 100 MHz by default; nexysa7 at 4) and the harness clock, which sets `STREAM_VCO_MHZ` / `STREAM_CLKOUT0_DIVIDE` and `HARNESS_CLK_HZ`; the bitstream name carries `clk<MHz>` when not 100
`CLKOUT0_DIVIDE`	board top	100, 80 or 60 MHz harness clock

Knob	Where	What it selects
	parameter	
`stream_env`	host	which build's `host/` layer goes on the Python path (default `mon`; perf sets it explicitly so a perf program cannot import mon's copy of a sibling)

6 Host Tools and Flow

6.1 The shared flow

The area Makefile is a dispatcher: `make bitstream BUILD=obs` runs the target in `build-obs/`, whose Makefile is variables plus an include of `make/fpga_flow.mk`. The flow's targets are therefore identical across the three builds and across the other Genesys 2 and Nexys A7 systems that use it.

Table 4.1: Flow targets (from `make/fpga_flow.mk`)

Target	What it does
`lint`, `flat-filelist`, `lint-decl-order`	Verilator lint of the whole harness against the expanded filelist, before any Vivado run
`prebuild`	the build's PREBUILD step: here, regenerate the bridges
`project`, `synth`, `bitstream`	create the Vivado project; synthesis only with utilization and failing-path reports; the full flow with every report
`bitstream-ila`	the same design plus an ILA on the marked debug nets
`sim`	this build's harness cosim (cocotb, Verilator)
`program`	flash the board, naming the file, falling back to the hold copy after a clean
`run SEQUENCES="..."`, `seq-list`	drive the programmed board through the area's sequences
`utilization`, `timing`	print the latest reports
`keep`	copy the bitstream to `RDS_HOLD_DIR` outside the repo and the reports to `../stable/`
`clean`, `clean-build`, `clean-all`	remove Vivado artifacts, cosim trees, everything; `clean-all` is what `stable/` protects against

6.2 The host layers

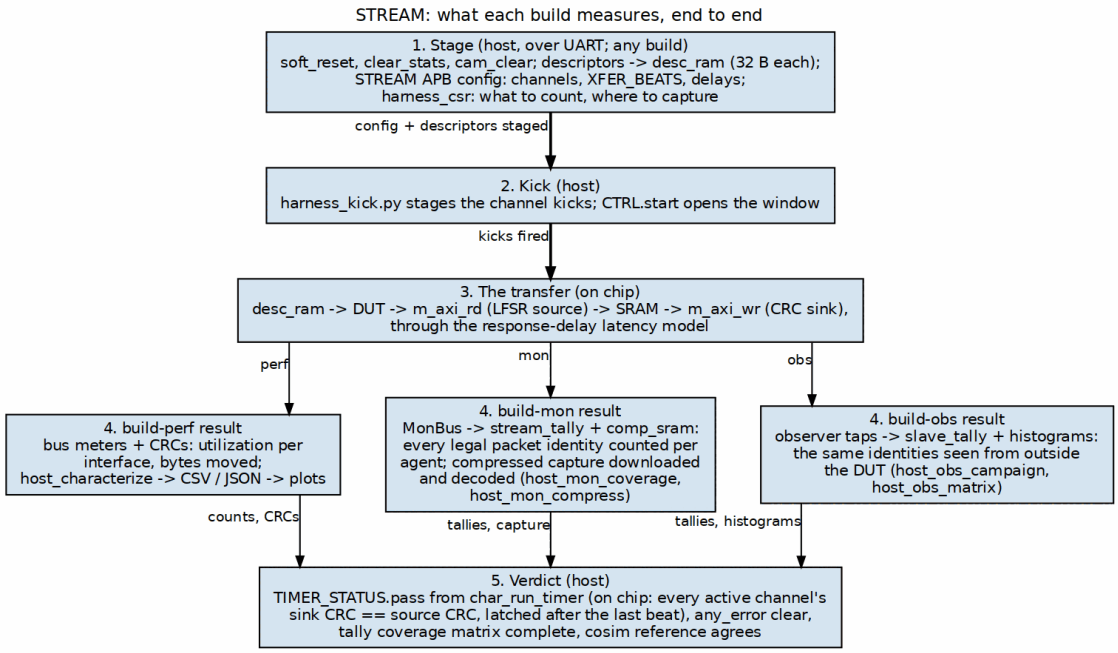
Two levels of Python. The component `bin/` holds what every build uses: `stream_device.py` (the UART link and register access by name), `harness_kick.py` (staging the channel kicks), `bus_meters.py` (the meter packing, one definition shared with the cosim so both measure the same thing), `tally.py` and `dump_monbus.py` (reading the tallies and decoding a capture), `characterization.py` and `stream_ext_suite.py` (the campaign runners), `stream_monitors.py` (monitor and observer configuration), and the plotting scripts. Each build then has its own `host/` with the programs that only make sense on that bitstream:

Table 4.2: Per-build host programs

Build	Programs
`perf`	`host_characterize`, `host_bus_meters`, `host_desc_perf`, `host_rw_perf`, `host_ext_char`, `host_ext_soak`, `host_probe_multichannel`, `host_verify_descriptors`, `host_status`, `host_perf_json_to_csv`
`mon`	`host_mon_coverage`, `host_mon_compress`, `host_mon_err_window`, `host_mon_matrix`, `host_mon_err_probe`, `host_mon_fault_probe`, `host_reg_walk`
`obs`	`host_obs_campaign`, `host_obs_matrix`, `host_reg_walk`

`stream_env` selects which build's `host/` goes on the Python path. Its default is `mon`; `build-perf` sets it explicitly because without that a perf program importing a sibling from its own directory would get mon's copy, and it would appear to work, since Python also puts the running script's own directory first.

6.2.1 Figure 4.1: What each build measures, end to end



The three measurement paths

Source: 04_measurement_path.dot

6.3 One program, two targets

The same host program runs against the cosim and against the board. The cocotb harness testbench presents the UART bytestream to `stream_harness` exactly as the FT232R does, the harness carries the bridge and the CSRs, and the generics are the build's. A monitor coverage result on the board is therefore compared with the same program's result in cosim, and the sign-off manifest records both (512 packets in 516 slots on the board against 1.12 slots per packet in cosim, for the compression program). A host program that only works on one target is a defect in the harness or the program, not a difference in kind.

The sign-off discipline before a build is the same on every flavour: the FULL-level sim suites of all three builds green on a verified-clean tree, recorded in the manifest with the tree SHA, so the bitstream is reproducible from the recorded source.

6.4 Where the results go

The STREAM / stream-g comparison has its own entry points: the point profiles under `build-perf/csv/` go to the board through `make host-characterize ARGS="--csv csv/<profile>.csv --json-output ..."` with `STREAM_G=0` and `1` programmed in turn, and through the harness simulation of each design with `make sim-gcompare | sim-gscale | sim-gscale1` (`dv/tests/test_stream_g_profiles.py`, the `char_csv` test type, results under `build-perf/results/sim/`); `bin/compare_profiles.py` puts the two designs, and the simulation against the board, side by side. Chapter 5 has the method and the numbers.

Table 4.3: Reports

Report	Source build	Content
`reports/perf`	perf	utilization sweeps: channels, descriptors, sizes, latency (the CSV data under `docs/data/`)
`reports/compression`	mon	MonBus compression on the board against the cosim
`reports/area`	any	what the monitors and the readback logic cost in LUTs per channel count
`reports/ext_addressing`	perf	the extended row/column addressing campaign
`stable/results`	mon	the board sweeps measured on the sign-off bitstream

Each report directory has its own README and, where a deliverable exists, the versioned DOCX and PDF built through the house pipeline.

7 STREAM against stream-g on This Board

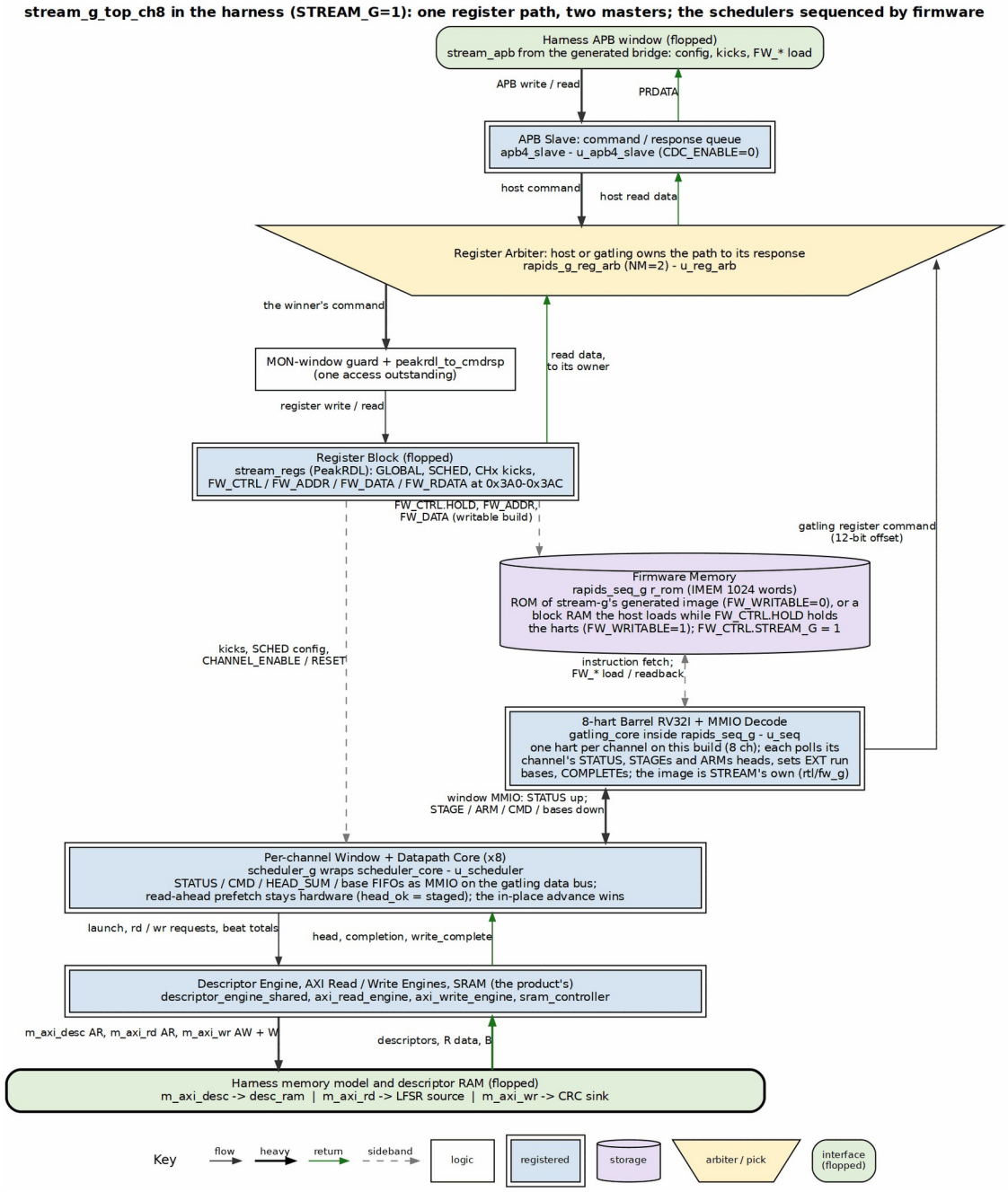
Last Updated: 2026-10-24

stream-g (STREAM MAS Chapter 7) keeps STREAM's descriptors, register map, engines and masters and replaces the per-channel scheduler state machines with firmware on one gatling core. This chapter is the board side of the comparison: the build that puts stream-g in the harness, what the two bitstreams cost, how the two designs were measured against each other, and every number measured. Which design to choose is STREAM HAS Chapter 5.4, which rests on the tables here; rapids HAS Chapter 6.4 is the same comparison for RAPIDS.

7.1 The stream-g build

`STREAM_G=1` on `build-perf` makes the DUT `stream_g_top_ch8` instead of `stream_top_ch8`; the harness is otherwise the same bitstream (the `STREAM_G` define reaches Vivado through `fpga/tcl/create_project.tcl`, lint through `LINT_DEFINES`, and the harness sim through `TEST_STREAM_G`). The bitstream name takes `_g`, `FW_WRITABLE=1` makes the firmware memory a block RAM the host loads through the `FW_CTRL` / `FW_ADDR` / `FW_DATA` / `FW_RDATA` registers (bitstream `_g_fw`) instead of a ROM of the generated image, and the host reads which design it has from `BUILD_CONFIG.STREAM_G` in the harness CSRs and `FW_CTRL.STREAM_G` in the STREAM map. `seq_restart` is tied low in the harness: the harts run from reset and the firmware polls. Figure 5.1 is the DUT's register path and sequencing structure as it sits in the harness; the engines, the SRAM and the three masters below it are the product's and are drawn in Figure 2.1.

7.1.1 Figure 5.1: The stream-g DUT in the harness



stream_g_top_ch8 in the harness: one register path, two masters

Source: 06_stream_g_dut.dot

For a board question that needs a waveform, the harness carries `mark_debug` marks under `STREAM_CHAR_ILA` on the host AXI4-Lite, the STREAM APB, the three DUT masters, the monitor

writer and the unit reset; `make bitstream-ila [STREAM_G=1]` builds the same design with an ILA on them (`fpga/tcl/build_ila.tcl`) and `make tcl-capture_ila` programs it, arms on the first DUT-master or host command and writes the capture as CSV (`fpga/tcl/capture_ila.tcl`). The marks cost the ordinary builds nothing.

7.2 The two bitstreams

Both from the same tree and the same flow, 8 channels, 100 MHz, built minutes apart on 2026-10-24:

Table 5.1: The two Genesys 2 builds

	STREAM (`stream_perf_genesys2_8ch.bit`)	stream-g (`stream_perf_genesys2_8ch_g.bit`, ROM firmware)
Slice LUTs	66,221 (32.5 %)	65,472 (32.1 %)
Slice registers	59,652	58,265
LUT as memory	6,390	6,742
Block RAM tiles	26	29
DSP48	64	0
Setup WNS / hold WHS (ns)	+0.790 / +0.032	+0.668 / +0.050
sha256	`ff9a6b95ff7d58f091e77acd23c5ee8c175d4451e72b75bd060664dc8889452a`	`b403115324623aa04d272c1710c5d0d35eec2bf157060d268300d0d9981687d1`

The 64 DSPs are the state machine's EXT run-base generators (a multiplier pair per channel per direction, `stream_run_addr_gen`); stream-g computes the bases in firmware and the gatling's own datapath is LUTs. The three extra block RAMs are the firmware ROM and the harts' scratch. The cores on their own, out of context at 8 and 16 channels in the board geometry (`fpga/tcl/ooc_stream_core.tcl`, `reports/ooc/`), are Table 5.2.

Table 5.2: The cores out of context on the xc7k325t-2 (128-bit data, 32-bit addresses, 8 outstanding each way, SRAM depth 256, shared engines, no AXI monitors, EXT addressing built; placed and routed at 100 MHz, 2026-10-24)

`stream_core` / `stream_core_g`, out of context	STREAM 8ch	stream-g 8ch	STREAM 16ch	stream-g 16ch
LUTs	32,025	32,164 (+0.4 %)	61,106	59,386 (-2.8 %)
LUT as distributed RAM	3,216	3,568	6,232	6,936
Registers	24,532	23,027	46,231	41,942

stream_core / stream_core_g, out of context	STREAM 8ch	stream-g 8ch	STREAM 16ch	stream-g 16ch
Block RAM tiles	16	19	32	35
DSPs	64	0	128	0
Setup WNS at 100 MHz (ns)	+0.581	+0.626	+0.256	+0.127
Hold WHS (ns)	+0.052	+0.051	+0.057	+0.051

In the board geometry the gatling and its windows cost about what the eight state machines and their run-base generators cost in LUTs (+0.4 % at 8 channels, -2.8 % at 16), fewer registers, three more block RAM tiles (the firmware memory and the harts' scratch) and no DSPs; both close 100 MHz at both channel counts, the 16-channel builds with the thinner margin. The register arbiter (`rapids_g_reg_arb`) sits in the top, outside `stream_core_g`, and is in Table 5.1 but not here.

7.3 The first stream-g bitstream

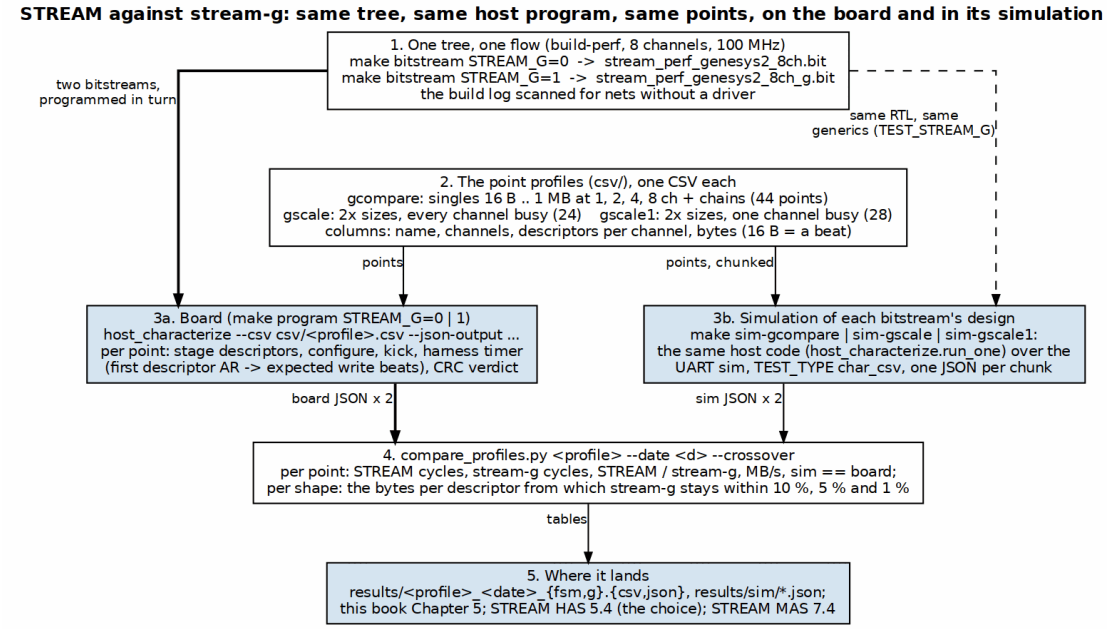
The first `STREAM_G=1` bitstream answered nothing on the UART, not even the harness's own CSRs, while a `STREAM_G=0` control built from the same tree answered on the same port. The Vivado log named it: `Net host_cmd_valid in module/entity stream_g_top_ch8 does not have driver`. The generated top had connected the APB slave to the `host_*` nets before the include that declares them; Verilator resolved the names and every lint and the harness sim were green, Vivado made implicit undriven nets, and the host register path did not exist in silicon. The generator anchors the include ahead of the APB slave now, and `make bitstream` prints every such line from the log after every build (the scan also surfaced the observers' undriven `w_comp_stat_*` nets, utility-ip misc BUG-002). The lesson is in the handbook's generated-RTL note; the control build is the A/B that settled it, and the bridge the build had regenerated was cleared the same way (its 1 MB matrix is cycle-identical to the 2026-10-03 one).

7.4 How the two designs were compared

Figure 5.2 is the method. The host program is `host_characterize.py`, the one every perf matrix on this board has run; a point is a configuration (channels, descriptors per channel, bytes per descriptor) that it stages, configures, kicks and times with the harness timer (first descriptor AR to the expected write-beat count, Section 2), with the CRC verdict on every channel that ran. The point profiles are rapids-g's three (`gcompare`, `gscale`, `gscale1`) in STREAM terms, as CSV inputs under `build-perf/csv/`: STREAM is memory to memory, so there is no sink, source or interleave split, and a beat is 16 bytes. Names read `beat_ch<N>_b<B>` (one descriptor per channel) and `chain_ch<N>_b<B>_d<D>` (a chain of D per channel), N active channels, B beats per descriptor. Every figure is bytes moved over the timer's cycles, so it includes the descriptor fetch and the launch; the peak is 1,600 MB/s.

The same points run on the harness simulation of each design through the same host code (`make sim-gcompare | sim-gscale | sim-gscale1`, the `char_csv` test type, one JSON per chunk under `results/sim/`), so the board can be checked against its simulation point for point. `bin/compare_profiles.py <profile> --date 2026-10-24 --crossover` tabulates both designs, marks the sim-against-board agreement, and derives the sizes at which stream-g is within 10 %, 5 % and 1 %.

7.4.1 Figure 5.2: The A/B, end to end



STREAM against stream-g: same tree, same host program, same points

Source: 07_ab_measurement.dot

7.4.2 The simulation against the board

The harness simulation of each bitstream's design, in the bitstream's own configuration (the perf geometry with `USE_AXI_MONITORS=0`; a first pass had taken the package default and built the in-core monitors in, which is not this bitstream), reproduces the board. The full three-profile campaign on both designs (`make sim-gcompare | sim-gscale | sim-gscale1`, 40 cells, `results/sim/`), on the 2026-10-25 bitstreams: on STREAM all 96 points are the board to the cycle; on stream-g 2 of 96 are identical and the rest differ by 16 cycles on average and 51 at most. That residue is the design's own: the firmware's poll loop runs free of the host, so where a hart's pass stands when a kick lands moves the count by up to a pass. On the first image the same campaign gave 24 cycles on average and 190 at most, one whole COMPLETE / NEXT_DESC walk, because that image's one-slot poll tested the chain hook only on a pass's first HEAD_SUM read and a chained head opened mid-pass by the in-place advance fell to JUDGE; the simulation trace of channel 0's scheduler (`bin/sched_trace_timeline.py`, `results/task021_trace_2026-10-24_g.jsonl`) is what found it, and STREAM's own firmware tests the hook on both reads (stream TASK-021: lone channel, 16-descriptor chains at 16 / 32 / 64 / 128 beats, board 704 / 1,126 / 1,319 / 2,199 cycles, simulation 688 / 1,132 / 1,312 / 2,196).

7.5 Single descriptor per channel (gcompare)

Table 5.3: One descriptor per channel, 2026-10-25 (peak 1,600 MB/s)

Point	ch	per descriptor	STREAM cycles	stream-g cycles	STREAM MB/s	stream-g MB/s	stream-g / STREAM
beat_ch1_b1	1	16 B	33	203	46	8	16.3 %
beat_ch1_b4	1	64 B	39	210	157	29	18.6 %
beat_ch1_b16	1	256 B	63	235	388	104	26.8 %
beat_ch1_b64	1	1 KB	111	262	880	373	42.4 %
beat_ch1_b256	1	4 KB	303	441	1,289	886	68.7 %
beat_ch1_b1024	1	16 KB	1,071	1,236	1,459	1,264	86.7 %
beat_ch1_b4096	1	64 KB	4,143	4,328	1,509	1,444	95.7 %
beat_ch1_b32768	1	512 KB	32,815	32,990	1,524	1,516	99.5 %
beat_ch1_b65536	1	1 MB	65,583	65,723	1,525	1,522	99.8 %
beat_ch2_b1	2	16 B	35	199	87	15	17.6 %
beat_ch2_b4	2	64 B	43	216	284	57	19.9 %
beat_ch2_b16	2	256 B	79	224	618	218	35.3 %
beat_ch2_b64	2	1 KB	175	332	1,116	588	52.7 %
beat_ch	2	4 KB	559	710	1,398	1,100	78.7 %

Point	ch	per descriptor	STREAM cycles	stream-g cycles	STREAM MB/s	stream-g MB/s	stream-g / STREAM
2_b256							
beat_ch2_b1024	2	16 KB	2,095	2,246	1,492	1,391	93.3 %
beat_ch2_b4096	2	64 KB	8,239	8,376	1,517	1,492	98.4 %
beat_ch2_b32768	2	512 KB	65,583	65,716	1,525	1,522	99.8 %
beat_ch2_b65536	2	1 MB	131,119	131,262	1,525	1,524	99.9 %
beat_ch4_b1	4	16 B	39	211	157	29	18.5 %
beat_ch4_b4	4	64 B	51	217	479	113	23.5 %
beat_ch4_b16	4	256 B	111	269	880	363	41.3 %
beat_ch4_b64	4	1 KB	303	435	1,289	898	69.7 %
beat_ch4_b256	4	4 KB	1,071	1,210	1,459	1,291	88.5 %
beat_ch4_b1024	4	16 KB	4,143	4,289	1,509	1,457	96.6 %
beat_ch4_b4096	4	64 KB	16,431	16,563	1,522	1,509	99.2 %
beat_ch4_b32768	4	512 KB	131,119	131,268	1,525	1,524	99.9 %
beat_ch4_b65536	4	1 MB	262,191	262,338	1,526	1,525	99.9 %

Point	ch	per descriptor	STREAM cycles	stream-g cycles	STREAM MB/s	stream-g MB/s	stream-g / STREAM
beat_ch8_b1	8	16 B	47	224	260	54	21.0 %
beat_ch8_b4	8	64 B	67	224	729	218	29.9 %
beat_ch8_b16	8	256 B	175	326	1,116	599	53.7 %
beat_ch8_b64	8	1 KB	559	698	1,398	1,119	80.1 %
beat_ch8_b256	8	4 KB	2,095	2,242	1,492	1,394	93.4 %
beat_ch8_b1024	8	16 KB	8,239	8,386	1,517	1,491	98.2 %
beat_ch8_b4096	8	64 KB	32,815	32,959	1,524	1,517	99.6 %
beat_ch8_b32768	8	512 KB	262,191	262,343	1,526	1,525	99.9 %
beat_ch8_b65536	8	1 MB	524,335	524,490	1,526	1,525	100.0 %

7.6 Descriptor chains (gcompare)

Table 5.4: Chains of descriptors per channel, 2026-10-25

Point	ch	chain	STREAM cycles	stream-g cycles	STREAM MB/s	stream-g MB/s	stream-g / STREAM
chain_ch1_b1_d16	1	16 x 16 B	212	685	115	36	30.9 %
chain_ch1_b64_d16	1	16 x 1 KB	1,071	1,313	1,459	1,190	81.6 %
chain_ch1_b1024_d16	1	16 x 16 KB	16,431	16,496	1,522	1,516	99.6 %
chain_ch1_b8192_d4	1	4 x 128 KB	32,815	32,876	1,524	1,521	99.8 %
chain_ch8_b1_d16	8	16 x 16 B	287	704	681	277	40.8 %
chain_ch8_b64_d16	8	16 x 1 KB	8,239	8,278	1,517	1,510	99.5 %
chain_ch8_b1024_d16	8	16 x 16 KB	131,119	131,155	1,525	1,525	100.0 %
chain_ch8_b8192_d4	8	4 x 128 KB	262,191	262,242	1,526	1,525	100.0 %

7.7 The size sweeps (`gscale`, `gscale1`)

Each cell is STREAM cycles / stream-g cycles (1.00 = parity). `single` is one descriptor per channel, `chain` a 16-descriptor chain per channel; “8 ch” has every channel busy, “1 ch” channel 0 alone.

Table 5.5: STREAM / stream-g cycles over the size sweep (- - = not in the sweep)

Beats	per descriptor	single, 8 ch	chain, 8 ch	single, 1 ch	chain, 1 ch
1	16 B	0.22	0.42	0.16	0.32
2	32 B	0.22	0.43	0.18	0.31
4	64 B	0.32	0.78	0.22	0.33
8	128 B	0.39	0.88	0.21	0.35
16	256 B	0.54	0.96	0.28	0.43
32	512 B	0.67	0.99	0.31	0.51
64	1 KB	0.79	0.99	0.40	0.83
128	2 KB	0.89	1.00	0.50	0.95
256	4 KB	0.93	1.00	0.70	0.99
512	8 KB	0.97	1.00	0.80	1.00
1024	16 KB	0.98	1.00	0.86	1.00
2048	32 KB	0.99	–	0.92	–
4096	64 KB	1.00	–	0.96	–
8192	128 KB	–	–	0.98	–
16384	256 KB	–	–	0.99	–
32768	512 KB	–	–	0.99	–
65536	1 MB	–	–	1.00	–

The smallest bytes per descriptor from which every larger point in the sweep stays within the bound:

Table 5.6: Bytes per descriptor for stream-g to be within 10 %, 5 % and 1 % of STREAM

Shape	Busy channels	Within 10 %	Within 5 %	Within 1 %
single	1 of 8	32 KB	64 KB	256 KB

Shape	Busy channels	Within 10 %	Within 5 %	Within 1 %
descriptor				
single descriptor	8 of 8	4 KB	8 KB	32 KB
16-descriptor chain	1 of 8	2 KB	2 KB	8 KB
16-descriptor chain	8 of 8	256 B	256 B	512 B

The difference is a fixed latency, not a rate: about 140 to 170 clocks once per channel between the kick and the first read (the firmware's poll, STAGE and ARM and the state walk, against the state machine's few cycles), and the firmware's reaction to each chained head after it, which STREAM's read-ahead prefetch, kept in hardware for a head the firmware has staged, overlaps with the running descriptor. With every channel busy the other channels' data hides the cost; alone, a descriptor pays it in full, which is why the one-channel bounds sit eight times higher. The first image's one-channel chain column had a plateau from 32 to 80 beats (about 106 cycles a descriptor whatever its length, 1,890 cycles at 32 beats against 559 for STREAM): its poll tested the chain hook only on a pass's first HEAD_SUM read, and a head opened by the in-place advance mid-pass fell to the slow path, so every second descriptor walked COMPLETE / NEXT_DESC. stream-g's own firmware tests the hook on both reads (stream TASK-021); the 32-beat chain is 1,126 cycles now and the column is monotonic. These tables are the 2026-10-25 measurement on that firmware; the STREAM side is cycle-identical to 2026-10-24 on all 96 points, and the first image's profiles are kept beside them.

7.8 The matrix at three descriptor sizes

`host_characterize.py`'s 40-config matrix (1, 2, 4, 8, 16 descriptors per channel at 1 to 8 channels) at 1 MB, 64 KB and 4 KB per descriptor, 240 runs, all passing on both designs: the stream-g bitstream of 2026-10-25 (STREAM's own firmware) against the STREAM bitstream of 2026-10-24, whose 1 MB matrix is cycle-identical to the 2026-10-03 measurement.

Table 5.7: Cycles, STREAM / stream-g (stream-g relative to STREAM)

per descriptor	1 desc, 1 ch	1 desc, 8 ch	16 desc, 1 ch	16 desc, 8 ch	mean of 40
1 MB	65,583 / 65,761 (+0.27 %)	524,335 / 524,467 (+0.03 %)	1,048,623 / 1,048,667 (+0.00 %)	8,388,655 / 8,388,686 (+0.00 %)	+0.02 %
64 KB	4,143 / 4,273 (+3.14 %)	32,815 / 32,957 (+0.43 %)	65,583 / 65,655 (+0.11 %)	524,335 / 524,369 (+0.01 %)	+0.30 %
4 KB	303 / 456 (+50.50 %)	2,095 / 2,253 (+7.54 %)	4,143 / 4,200 (+1.38 %)	32,815 / 32,844 (+0.09 %)	+4.72 %

7.9 Artifacts

What	Where (under `build-perf/`)
The point profiles	`csv/gcompare.csv`, `csv/gscale.csv`, `csv/gscale1.csv`; `csv/task021_probe.csv` (the finer grid around the chain plateau)
Board runs	`results/{gcompare,gscale,gscale1}_2026-10-25_{fsm,g}.{csv,json}` (STREAM's own firmware), `..._2026-10-24_...` (the first image); `results/perf_sweep_2026-10-{24,25}_{fsm,g}_{1MB,64KB,4KB}.{csv,json}`; `results/task021_probe_2026-10-{24,25}_g.{csv,json}`
Simulation runs	`results/sim/<profile>_<dut>_<k>of<n>.json`
Out-of-context cores	`../reports/ooc/<top>_<ch>ch/10.0ns/`
Tabulation	`../bin/compare_profiles.py`